Embedded Systems

Secure Software & Hardware Systems

Overview

- Embedded Systems
  - Definition
  - Characteristics
  - Components
- Protocols & Interfaces
  - UART
  - SPI
  - JTAG
  - I2C
- Recap

Overview

- Embedded Systems
  - Definition
  - Characteristics
  - Components
- Protocols & Interfaces
  - UART
  - SPI
  - JTAG
  - I2C
- Recap

Embedded Systems are …

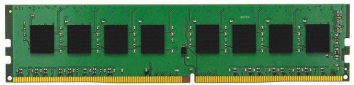
Everywhere:












Embedded Systems are ...

Evermore connected:

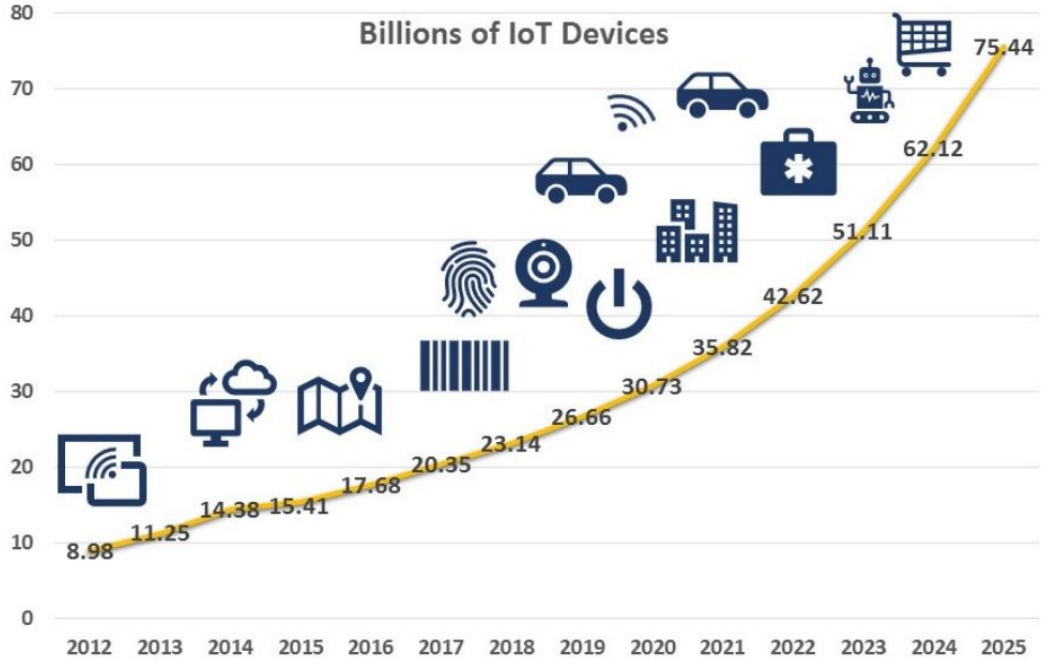


Image source: Toor et al. "*Combined Access Barring Scheme for IoT Devices Using Bayesian Estimation*." Electronics 9.12 (2020).
Based on data available at: https://www. statista.com/statistics/471264/iot-number-of-connected-devices-worldwide/

Embedded Systems are …

Everlasting source of news:

Why analyzing embedded devices?

Why embedded?

- Embedded systems pervade nearly every aspect of modern life
- Typically, those systems are required to be functioning

Why analyzing/fuzzing/exploiting?

- State of security often lags behind
- Interesting hardware/software interactions
- Different attack scenarios than for traditional computing systems

Overview

- Embedded Systems
  - Definition
  - Characteristics
  - Components
- Protocols & Interfaces
  - UART
  - SPI
  - JTAG
  - I2C
- Recap

Definition

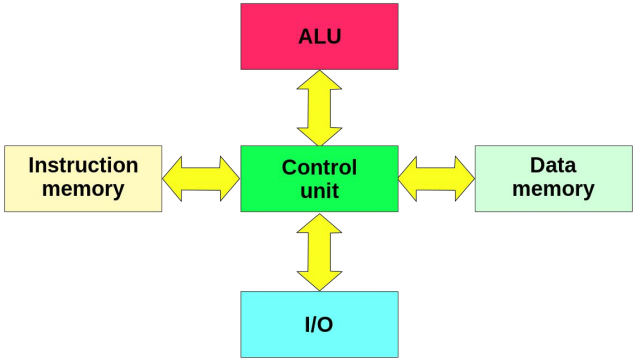
Embedded Systems:

- are special-purpose computing systems
- running software tightly coupled to the hardware
- as part of a larger system

Characteristics

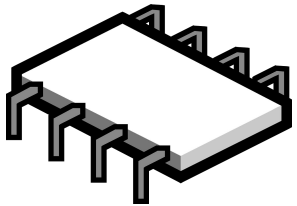
Embedded Systems *may* have:

- No, or specialized, user interfaces
- Low power consumption
- Low computational power
- Interfaces to the physical world
- Non-customizable software
- A non-von Neumann architecture

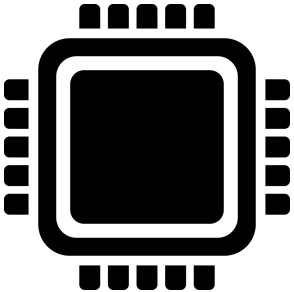


Harvard Architecture

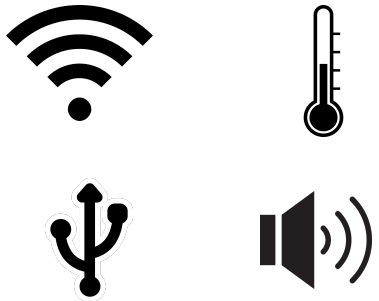
Core Components



Memory



Processing Unit

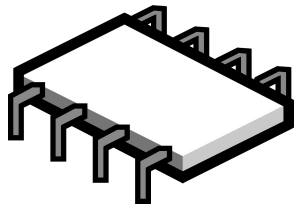


Peripherals

Memory (non-volatile)

- Typically holds
  - Code
  - Static Data
  - Configuration

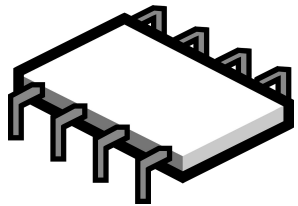
- Common types:
  - Masked Read-Only Memory (Mask ROM)
  - Erasable Programmable Read-Only Memory (EPROM)
  - Electrically Erasable Programmable Read-Only Memory (EEPROM)
  - NAND/NOR Flash



Memory (volatile)

- Typically holds
  - Run-time data (stack, heap, non-ro data)
  - Code (copied from NV-memory)

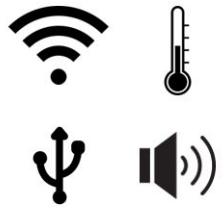
- Common types:
  - Dynamic Random-Access Memory (DRAM):
    - As known from desktop PCs
  - Static Random-Access Memory (SRAM):
    - Stores information in transistor logic
    - No refresh cycles required
    - Less power consumption than DRAM
    - More expensive than DRAM
    - More common for embedded systems



Peripherals

- Input/output devices

- Typically interfaced via
  - Memory-Mapped IO (MMIO)
  - Interrupts
  - Direct Memory Access (DMA)



More in next lecture

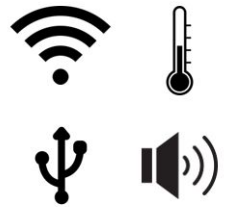
On-Chip vs Off-Chip Peripherals

On-Chip:

- Shares chip with processing unit
- Directly interfaced
- Usually simple, but essential peripherals
- Example: Timer

Off-Chip:

- Physical separation from processing unit
- Connected via bus
- May be embedded system on its own
- Example: WiFi Chips



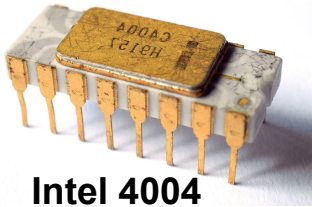
Processing Unit

Simplest case: Microprocessor

- Integrated Circuit (IC) which holds CPU & Peripheral/Memory interfaces

CPUs on (most) embedded systems:

- Low complexity
- Low power consumption
- Workload specific instruction set architecture (ISA)



Intel 4004

Processing Unit - ISAs

- More variety than for desktop computers
  - ISAs are often tied to specific vendors
  - Most are reduced instruction set computer (RISC) architectures
- Some examples:

MIPS

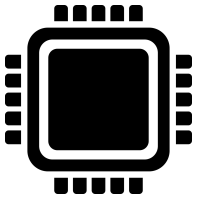
PowerPC

AVR®

SPARC

arm

Processing Unit - Variants



- Microcontroller (MCU):
  - IC containing also memory and peripherals
- System-on-Chip (SoC):
  - IC containing a full computing system
- Digital signal processor (DSP):
  - Microprocessor specialized for digital signal processing
- Coprocessor:
  - Microprocessor supporting a main processor for specialized tasks

Exercise: Identify Components

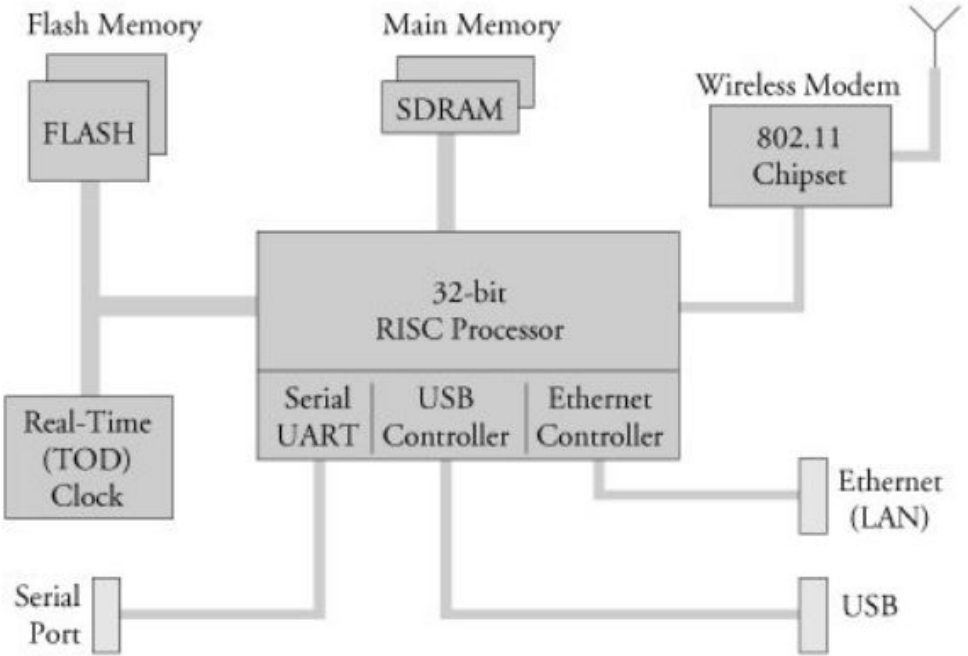


Image source: C. Hallinan. "Embedded Linux Primer: A Practical, Real-World Approach"

Exercise: Identify Components

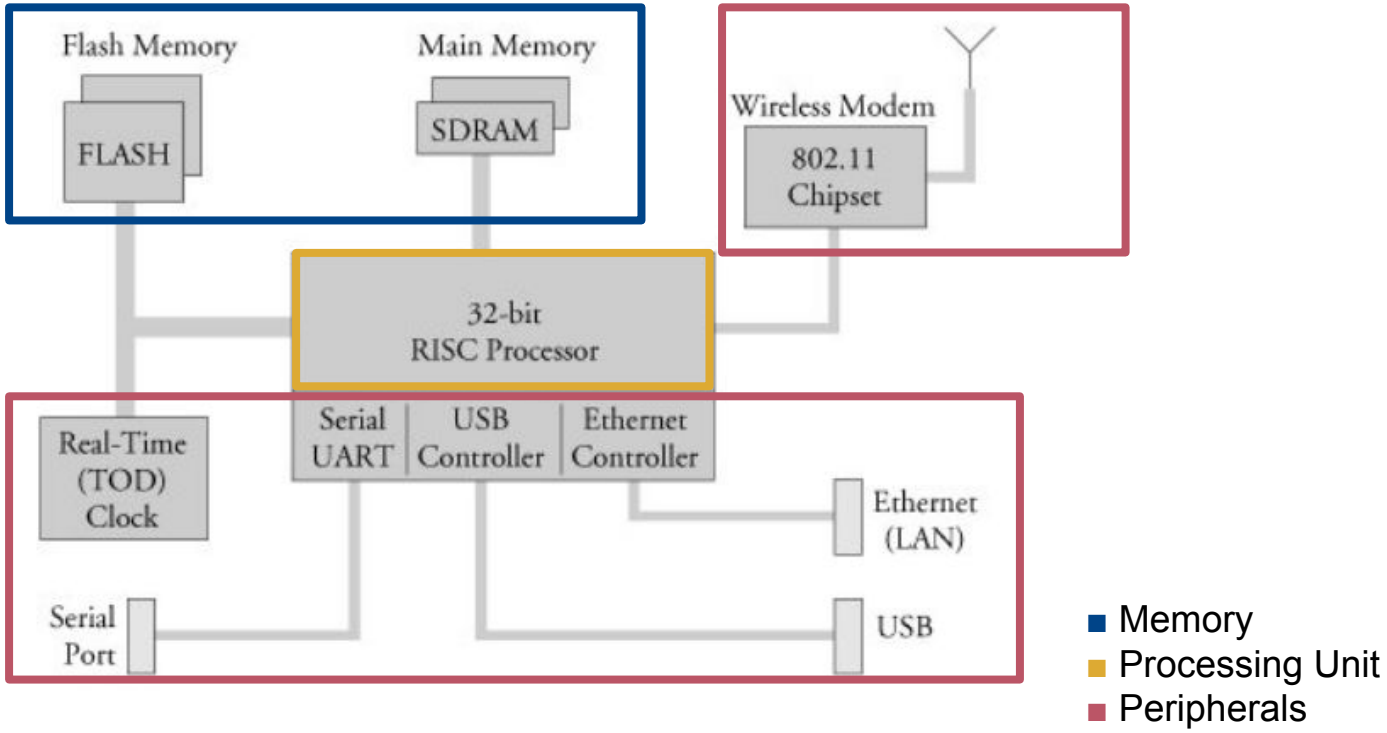


Image source: C. Hallinan. "Embedded Linux Primer: A Practical, Real-World Approach"

Overview

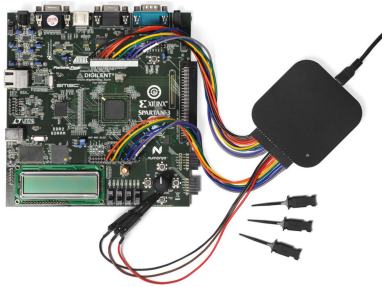
- Embedded Systems
  - Definition
  - Characteristics
  - Components
- Protocols & Interfaces
  - UART
  - SPI
  - JTAG
  - I2C
- Recap

Inspection Tools

Multimeter

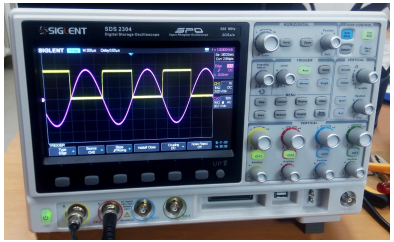
- Measuring of:
  - Voltage
  - Current
  - Resistance
- Continuity Test

Logic Analyzer



- Measuring and visualizing digital signals
- May even provide decoder

Oscilloscope



- Measuring and visualizing analog signals over time

Protocols & Interfaces

Wired:

- I^2C
- SPI
- CAN
- UART
- Modbus
- JTAG
- SWD
- ...

Wireless:

- Bluetooth
- Zigbee
- WiFi
- GSM
- CC1xxx
- ...

Protocols & Interfaces

Wired:

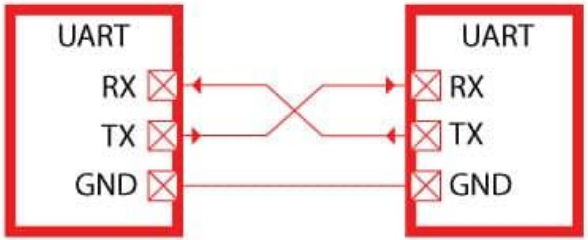
- I²C
- SPI
- CAN
- UART
- Modbus
- JTAG
- SWD
- ...

Wireless:

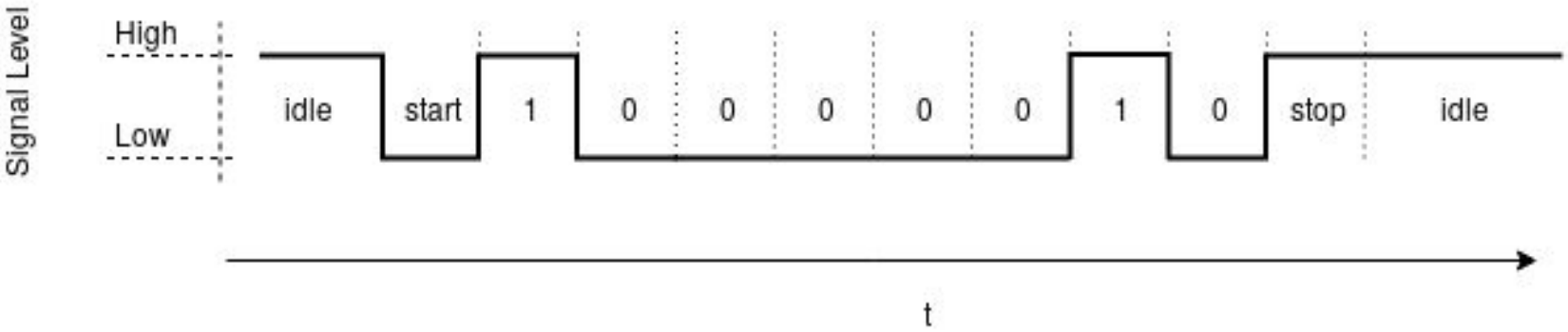
- Bluetooth
- Zigbee
- WiFi
- GSM
- CC1xxx
- ...

Universal Asynchronous Receiver/Transmitter (UART)

- Used for serial communications between two components
  Separate receive (RX) and transmit (TX) lines:
  - TX chip1 -> RX chip2
  - TX chip2 -> RX chip1
- Configurable data format and transmission speed
  Agreement on parameters required beforehand



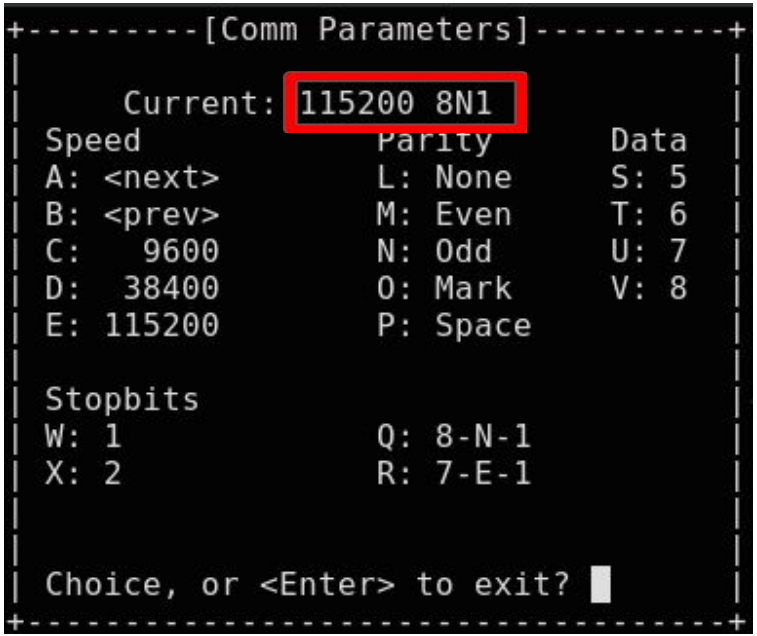
UART – Electrical Signal



UART – Configuration

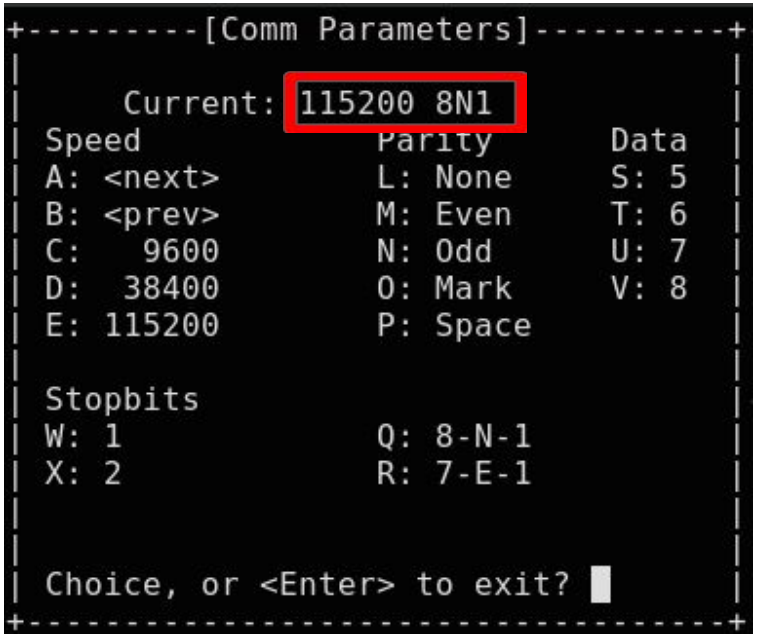
Options & common choices:

- Baud rate:
  - 2400 - 4800 – 9600 – 19200 – 38400 – 57600 – 115200
- Data bits:
  - 5 – 6 – 7 – 8
- Parity bit:
  - None - Even - Odd
- Stop bit:
  - 1 – 1.5 – 2
- Bit order:
  - LSB first – MSB first

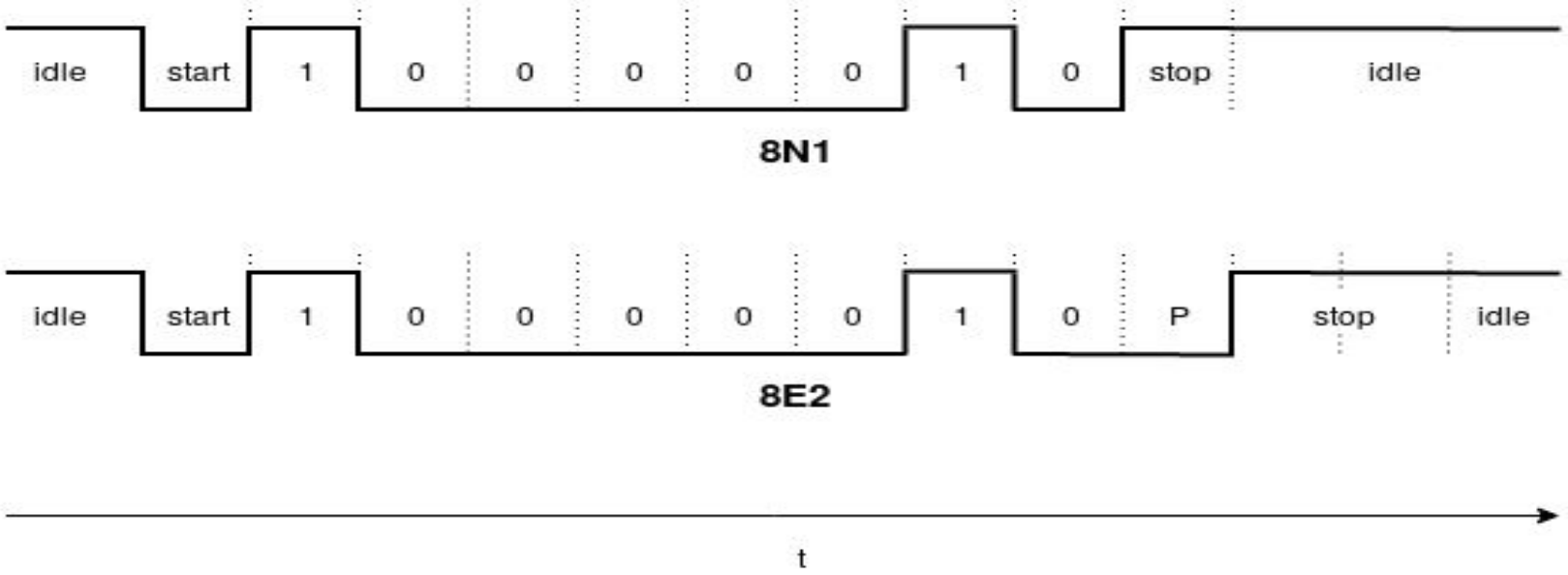


UART – Configuration

- Baud rate determines the transmission speed for UART communication
  - As there is no clock signal, both parties need to agree on it beforehand
- Formally, baud rate is defined as *symbols per second*
- Example: 115200 baud, 8 data bits, 1 stop bit, no parity (8N1):
  - 115200 baud → 115200 bits/second
  - 10 bits needed per byte: (1 start + 8 data + 1 stop bit)
  - → 115200 byte/s



UART – Configuration



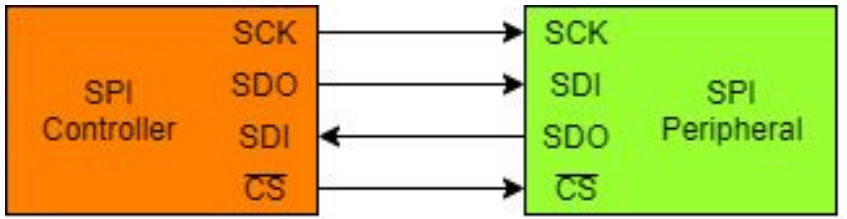
Discovering UART Ports

1. If available, consult datasheet
2. Locate promising headers
3. Identify grounded pins
   - continuity test
4. Identify TX pin
   - If data transmission is enabled fluctuating voltage is observable
5. Identify RX pin
   - Most cumbersome, may require connection to all possible RX pins

Image source + more information:
https://web.archive.org/web/20210522202158/devttys0.com/2012/11/reverse-engineering-serial-ports/

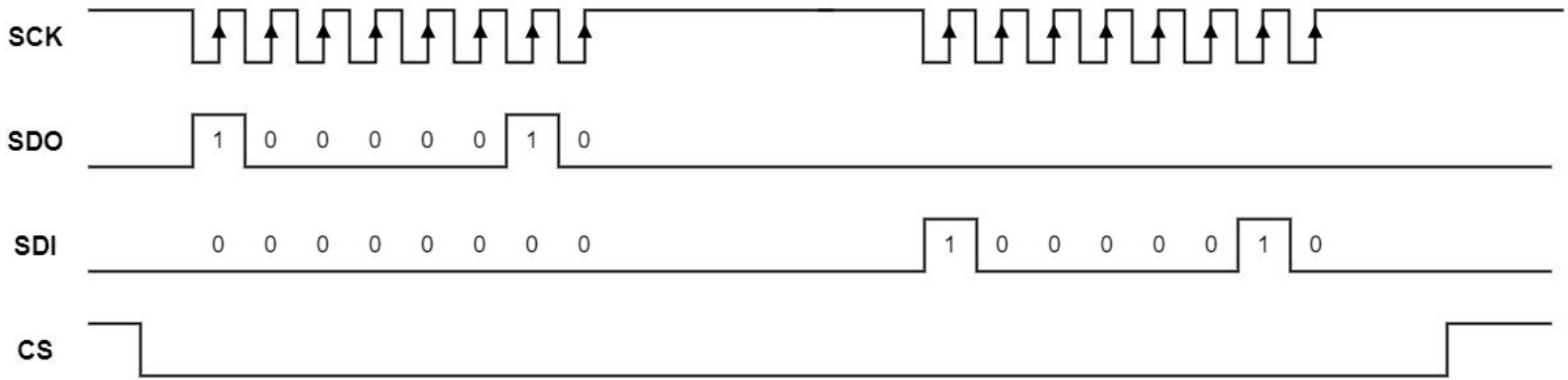
Serial Peripheral Interface (SPI)

- Synchronous serial bus protocol
- Connects two or more components on a bus
- Controller/Peripheral architecture
- Uses four lines:
  - 2x data
  - 1x clock
  - 1x chip select



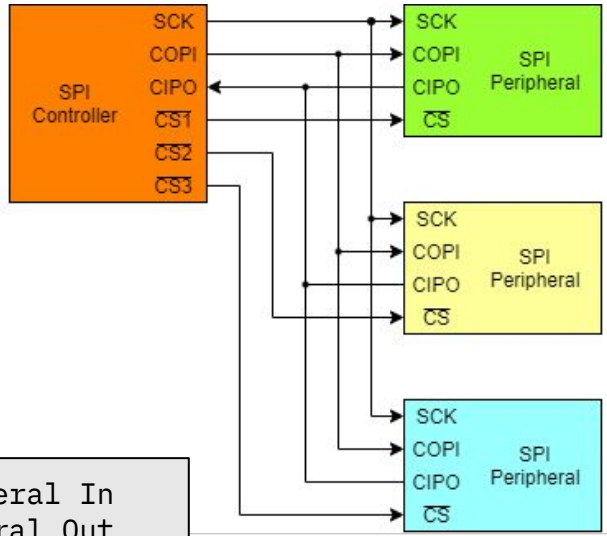
SPI – Protocol

SCK: Serial Clock
SDO: Serial Data Out
SDI: Serial Data In
CS: Chip Select

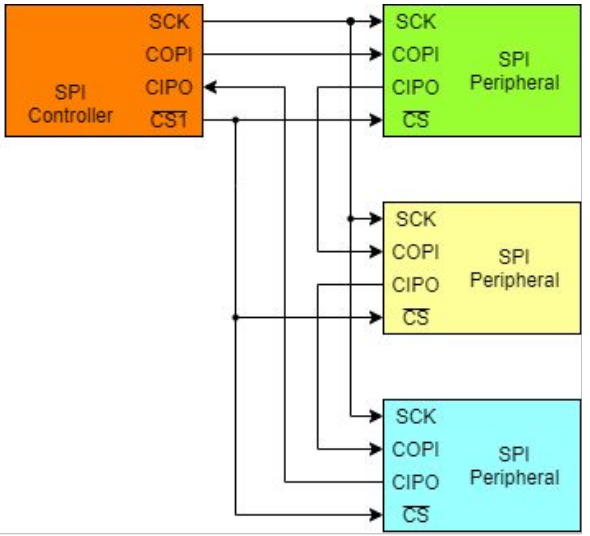


SPI – Multi Peripheral Configurations

Independent peripherals



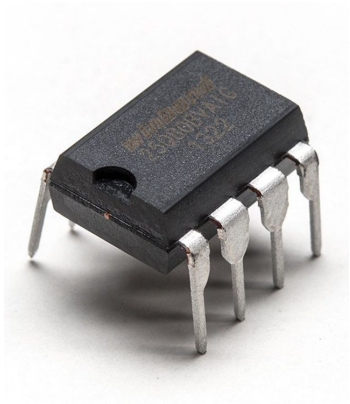
Daisy chained peripherals




```
COPI: Controller Out/Peripheral In
CIPO: Controller In/Peripheral Out
```

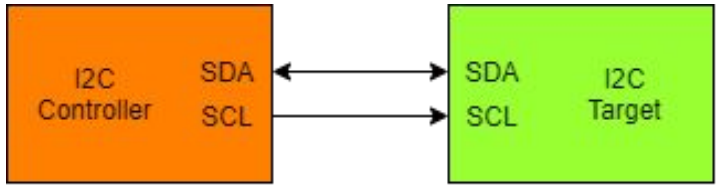
Discovering SPI flashes

- Typical device on SPI bus: Flash memory
  - E.g., the place where BIOS is stored
- Most often visible by eye
  - Manufacturer + part ID very likely printed on top of the chip
  - Datasheet can often be found!
- Open source tooling for SPI flash dumping available
  - E.g., https://trmm.net/SPI_flash



Inter-Integrated Circuit (I2C)

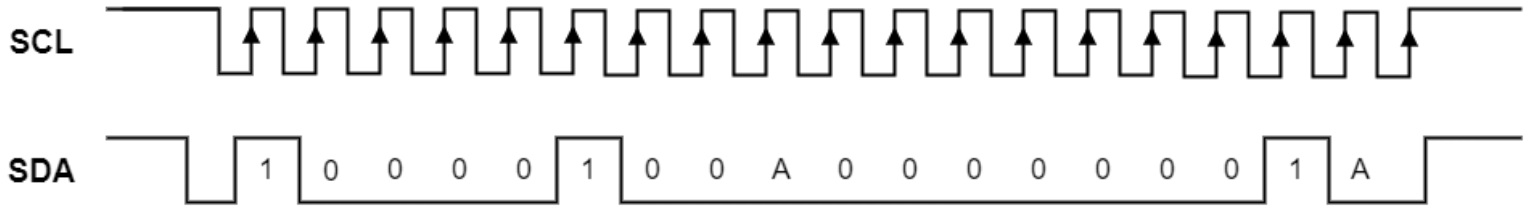
- Synchronous serial bus protocol
- Connects two or more components on a bus
- Multi-controller/multi-target architecture
- Uses two lines:
  - Serial Data (SDA)
  - Serial Clock (SCL)



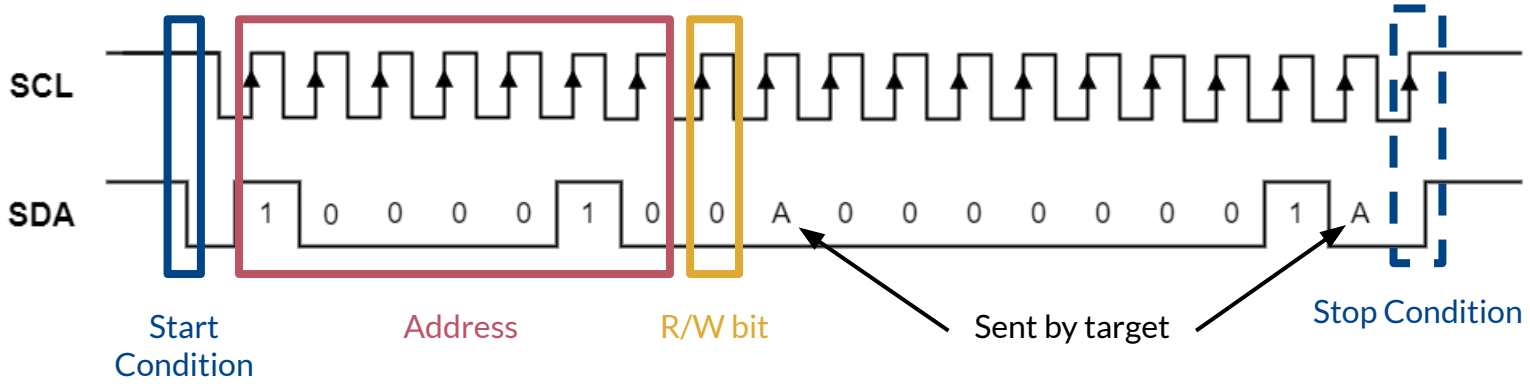
I2C Message

- Start Condition (S)
  - Signalizes begin of message
  - SDA pulled low while SCL high
- Address (7 or 10 bits)
  - Determines target to communicate with
- Read/Write bit (R/W):
  - 1: Read from target
  - 0: Write to target
- ACK/NACK bit (A):
  - Transmitter releases SDA
  - Receiver pulls line low
  - Transmitter reads stable low ACK
- Data (8 bits)
  - Actual Payload
- Stop Condition (P)
  - Signalizes end of message
  - SDA pulled high while SCL high

I2C Physical Transmission

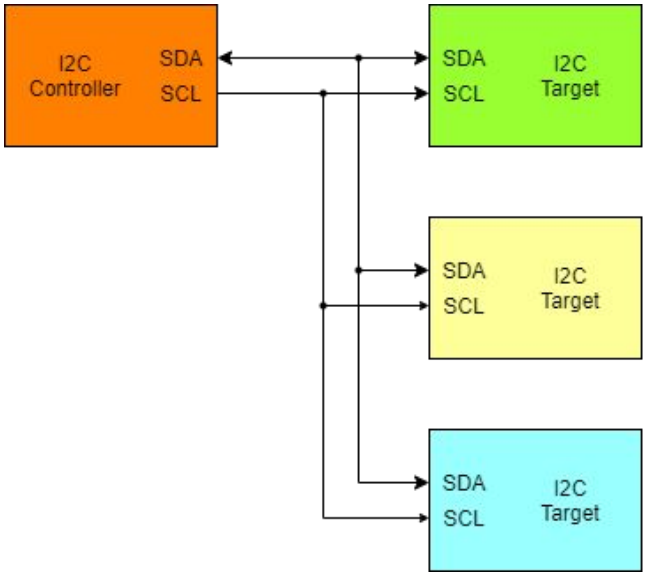


I2C Physical Transmission

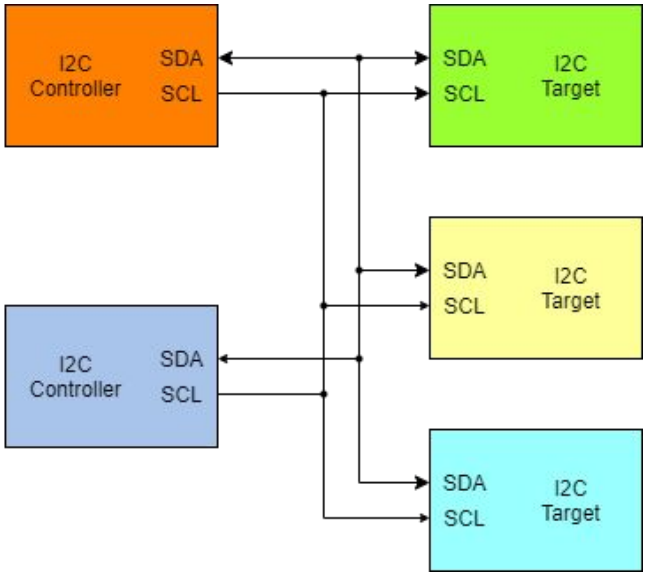


I2C - Multi-Target Architectures

Single-Controller/Multi-Target

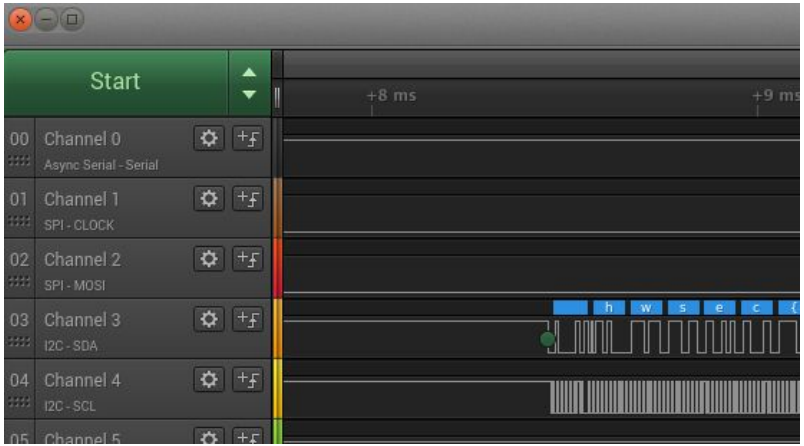


Multi-Controller/Multi-Target



Discovering I2C bus

1. If available, consult datasheet
   - Look for part IDs on the components!
2. Find likely I2C pins
   - Use multimeter to test connection between components
3. Attach Logic Analyzer/Oscilloscope
   - Check for characteristic I2C communication



SPI & I2C - Deprecated Terminology

- Terminology changed recently!
  - You are very likely to encounter the deprecated terms
  - Hence: here a summary of previously used terms

SPI

Term	Deprecated Term
Controller	Master
Target	Slave

I2C

Term	Deprecated Term
Controller	Master
Peripheral	Slave
Controller Out / Peripheral In (COPI)	Master Out / Slave In (MOSI)
Controller In / Peripheral Out (CIPO)	Master In / Slave Out (MISO)
Chip Select	Slave Select

More information: I2C: NXP Semiconductors. "I2C-bus specification Rev 7". October 1, 2021
SPI: https://www.oshwa.org/a-resolution-to-redefine-spi-signal-names

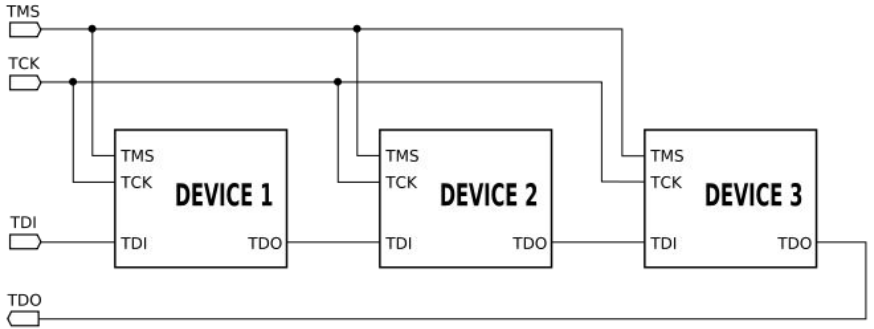
Joint Test Action Group (JTAG)

- Standardized Debugging Interface
  - Present on almost every prototyping board
  - Sometimes on production hardware, too!
- Specified in IEEE 1149.1 and defines:
  - The stateful protocol
  - Presence of instruction register (IR) & data registers (DR)
  - Minimum set of requireds JTAG instructions
    - Mandatory: BYPASS, EXTEST, SAMPLE/PRELOAD
    - Optional: IDCODE, INTEST
- Vendors commonly extend with custom instructions

JTAG Test Access Port (TAP)

- At least 4 lines:
  - TDI: Test Data In
  - TDO: Test Data Out
  - TCK: Test Clock
  - TMS: Test Mode Select
  - TRST: Target Reset (optional)

- Devices are arranged in a chain



JTAG State Machine

- Driven by TMS
- Two primitive operation:
  - Scan IR
  - Scan DR
- Data Registers:
  - Boundary Scan Register (BSR)
  - BYPASS
  - IDCODE
- Used data register dependent on instruction

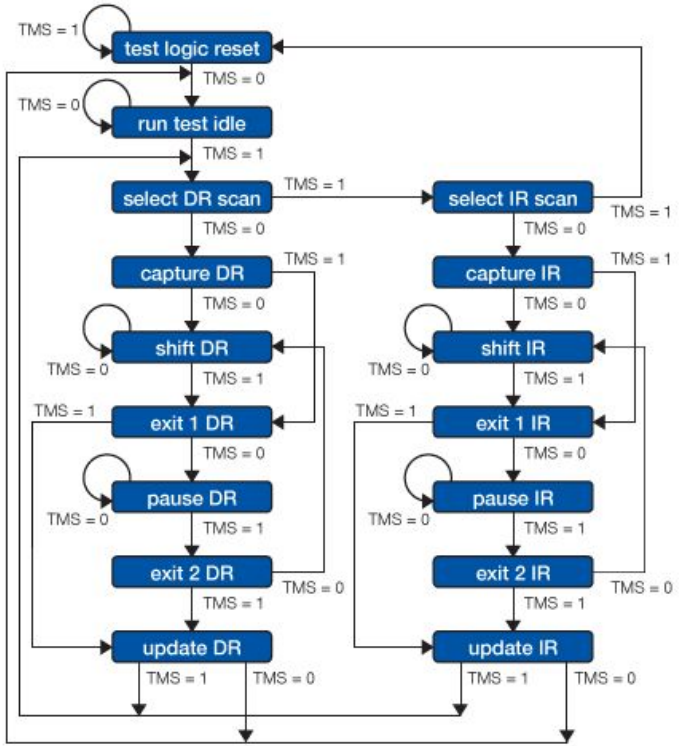


Image source: https://www.xjtag.com/about-jtag/jtag-a-technical-overview/

JTAG Protocol Example

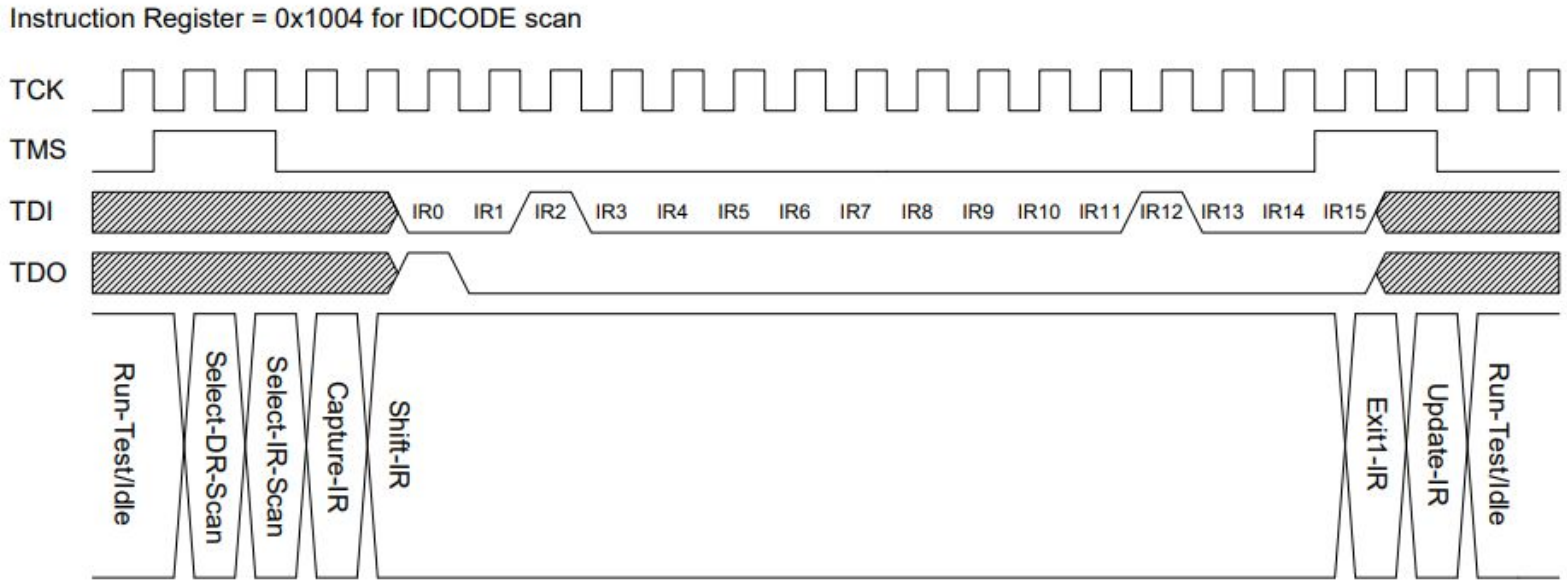
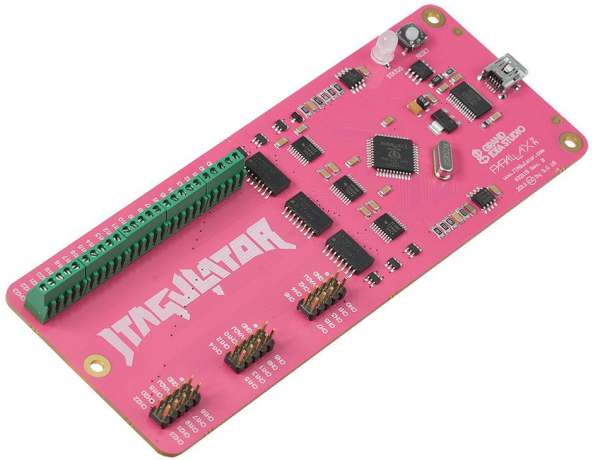


Image source: Silicon Labs. “AN105: Programming Flash Through The JTAG Interface (Rev. 1.4 12/03)”

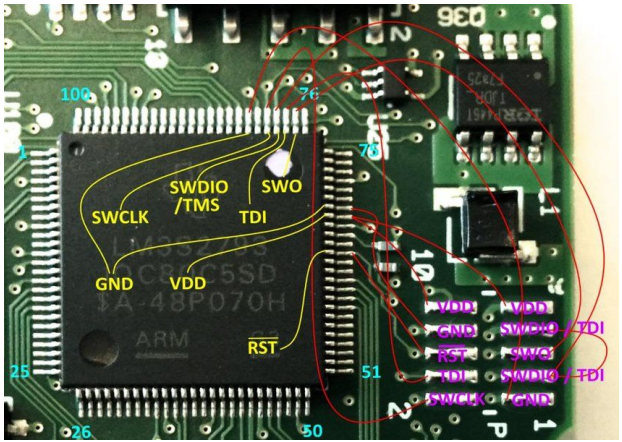
Discovering JTAG Ports

- If available, consult datasheet
- Locate promising headers
- Example: Bypass Scan
  a. Guess Pin configuration
  b. Continuously send “1”s on assumed TDI
  c. Send BYPASS instruction to TAP (set IR to all “1”s)
  d. Wait to receive “1”s on assumed TDO
  e. It not successful, repeat with other pin configuration

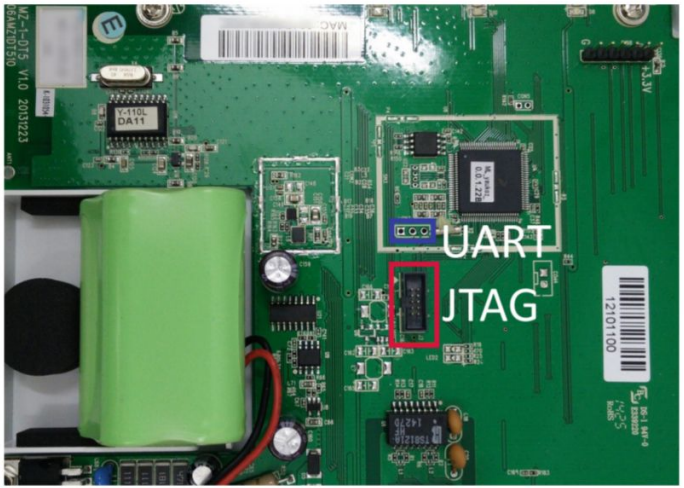
- Automated discovery tools:
  - JTAGulator
  - JTAGenum
  - Glasgow



JTAG Ports: Examples



L. A. Garcia et al. “Hey, My Malware Knows Physics! Attacking PLCs with Physical Model Aware Rootkit.” NDSS’18



S. Vasile et al. “Breaking All the Things – A systematic Survey of Firmware Extraction Techniques for IoT Devices.” CARDIS’18

Overview

- Embedded Systems
  - Definition
  - Characteristics
  - Components
- Protocols & Interfaces
  - UART
  - SPI
  - JTAG
  - I2C
- Recap

Lecture Recap

- Intro to embedded systems
  - "Why" are we doing this?

- Definition, characteristics, and components
  - "What" are we dealing with?

- Example protocols & interfaces
  - "How" do embedded systems communicate?